Manual de Instalación — Infraestructura n8n

Positiva / Artics — Contrato N° 0420-2026

⚙️ Variables de infraestructura

> *Antes de ejecutar cualquier comando o script de este manual, reemplazar los siguientes placeholders con las IPs reales del entorno. Se usan consistentemente en todo el documento.*

Placeholder	Rol	Ejemplo de valor
`<IP_LB>`	Load Balancer (Nginx)	`10.115.25.101`
`<IP_MAIN1>`	n8n Main 1 (nodo activo 1)	`10.114.158.71`
`<IP_MAIN2>`	n8n Main 2 (nodo activo 2)	`10.114.158.72`
`<IP_REDIS>`	Redis (cola de trabajos)	`10.114.158.75`
`<IP_WORKER>`	n8n Worker (ejecutor)	`10.114.158.76`
`<IP_POSTGRES>`	PostgreSQL (base de datos)	`10.114.158.77`
`<IP_MICRO>`	Microservicios (equipo ext.)	`10.114.158.86`
`<SUBNET_N8N>`	Subred interna de los nodos	`10.114.158.0`

Índice

1. Inventario de servidores
2. Prerrequisitos
3. Instalación paso a paso
   - 3.1 Redis
   - 3.2 PostgreSQL
   - 3.3 n8n Main 1
   - 3.4 n8n Worker
   - 3.5 n8n Main 2
   - 3.6 Load Balancer — Nginx HTTPS
4. Despliegue automatizado
5. Variables de entorno críticas
6. Bugs encontrados y fixes aplicados
7. Verificación post-instalación
8. Comandos de operación diaria

---

1. Inventario de servidores

Rol	Hostname	IP	CPU	RAM	Disco	SO
Load Balancer	n8n-lb		4 vCPU	4 GB	80 GB SSD	Ubuntu 24.04 LTS
n8n Main 1	n8n-nodo1		4 vCPU	8 GB	80 GB SSD	Ubuntu 24.04 LTS
n8n Main 2	n8n-nodo2		4 vCPU	8 GB	80 GB SSD	Ubuntu 24.04 LTS
Redis	n8n-redis		4 vCPU	6 GB	40 GB SSD	Ubuntu 24.04 LTS

Worker	n8n-worker		8 vCPU	8 GB	120 GB SSD	Ubuntu 24.04 LTS
PostgreSQL	n8n-postgres		6 vCPU	8 GB	200 GB SSD	Ubuntu 24.04 LTS
Microservicios	n8n-micro		12 vCPU	24 GB	500 GB SSD	Ubuntu 24.04 LTS

Acceso SSH general:

```
Usuario:    administrador
Contraseña: Temporal01#
Nota:       El usuario puede ejecutar sudo
```

---

2. Prerrequisitos

En la máquina desde donde se ejecuta el despliegue (WSL / Linux / Git Bash)

```
# Instalar sshpass (necesario para los scripts .sh)
sudo apt-get install -y sshpass

# Instalar Node.js (necesario para deploy-nodes.js y los fix-*.js)
# Versión recomendada: 18 LTS o superior
node --version

# Instalar dependencias del proyecto
cd /ruta/al/repositorio
npm install
# Esto instala: ssh2 ^1.17.0
```

> ***Nota importante:*** *Los scripts* `.sh` *(deploy-all.sh y los deploy.sh individuales) requieren ejecutarse en un entorno Linux/Unix (WSL, Git Bash o Linux nativo). Los scripts* `.js` *funcionan desde cualquier entorno donde esté Node.js disponible.*

En cada servidor destino (se verifica/instala automáticamente)

- Docker Engine — los scripts lo instalan automáticamente si no está presente
- Docker Compose plugin (viene incluido con Docker Engine moderno)

No se requiere ninguna instalación manual previa en los servidores.

---

3. Instalación paso a paso

El **orden de instalación es obligatorio**. Cada servicio depende del anterior. No instalar n8n antes de que Redis y PostgreSQL estén operativos.

Orden correcto:

```
Redis → PostgreSQL → n8n Main 1 → n8n Worker → n8n Main 2 → Nginx LB
```

3.1 Redis — `<IP_REDIS>:6379`

Redis actúa como la cola de trabajos (Bull Queue). Debe estar disponible antes que cualquier instancia de n8n.

Contenedor desplegado (`Redis/docker-compose.yml`):

```yaml
services:
  redis:
    image: redis:7-alpine
    container_name: n8n-redis
    restart: always
    network_mode: host
    command: >
      redis-server
      --bind 0.0.0.0
      --protected-mode no
      --save 60 1
      --loglevel notice
    volumes:
      - redis_data:/data
```

Puntos clave de la configuración:

- `--bind 0.0.0.0` — acepta conexiones desde cualquier IP (necesario para que Main 1, Main 2 y Worker se conecten)
- `--protected-mode no` — deshabilita el modo protegido (la red interna privada es el control de acceso)
- `--save 60 1` — persistencia: guarda snapshot cada 60 segundos si hubo al menos 1 cambio
- `network_mode: host` — el contenedor usa la IP real del servidor, no una red Docker bridge

Opción A — Usando el script automatizado (requiere sshpass en WSL/Linux):

```bash
cd Redis/
bash deploy.sh
```

Opción B — Instalación manual directa en el servidor (cuando no se puede usar deploy.sh):

```bash
ssh administrador@<IP_REDIS>

# 1. Instalar Docker si no está presente
if ! command -v docker &> /dev/null; then
    curl -fsSL https://get.docker.com | sudo sh
    sudo systemctl enable docker
    sudo systemctl start docker
fi

# 2. Crear directorio de trabajo
sudo mkdir -p /opt/n8n-redis
sudo chown administrador:administrador /opt/n8n-redis

# 3. Crear el archivo docker-compose.yml
cat > /opt/n8n-redis/docker-compose.yml << 'EOF'
services:
  redis:
    image: redis:7-alpine
    container_name: n8n-redis
    restart: always
    network_mode: host
    command: >
      redis-server
      --bind 0.0.0.0
      --protected-mode no
      --save 60 1
```

```
      --loglevel notice
    volumes:
      - redis_data:/data

volumes:
  redis_data:
    name: n8n_redis_data
EOF

# 4. Detener servicio systemd previo si existe
sudo systemctl stop redis-server 2>/dev/null || true
sudo systemctl disable redis-server 2>/dev/null || true

# 5. Levantar el contenedor
cd /opt/n8n-redis
sudo docker compose pull
sudo docker compose up -d

# 6. Verificar
sleep 3
sudo docker exec n8n-redis redis-cli ping
```

Verificación:

```
ssh administrador@<IP_REDIS>
sudo docker exec n8n-redis redis-cli ping
# Debe responder: PONG

sudo docker compose -f /opt/n8n-redis/docker-compose.yml ps
```

3.2 PostgreSQL — `<IP_POSTGRES>:5432`

Almacena todos los datos de n8n: workflows, credentials, executions, users.

> ⚠️ ***PRECAUCIÓN:*** *Si ya existe una instalación previa de PostgreSQL con datos, el script* `deploy.sh` *de Postgres realiza un backup automático (* `pg_dump` *) antes de continuar. Nunca saltar este paso.*

Contenedor desplegado (`Postgres/docker-compose.yml`):

```yaml
services:
  postgres:
    image: postgres:16-alpine
    container_name: n8n-postgres
    restart: always
    network_mode: host
    environment:
      POSTGRES_DB: n8n_db
      POSTGRES_USER: n8n_user
      POSTGRES_PASSWORD: PostgresSQL
      POSTGRES_HOST_AUTH_METHOD: md5
      POSTGRES_INITDB_ARGS: "--auth-host=md5 --auth-local=md5"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./pg_hba.conf:/etc/postgresql/pg_hba.conf:ro
    command: >
```

```
      postgres
      -c listen_addresses=*
      -c hba_file=/etc/postgresql/pg_hba.conf
```

Control de acceso (`Postgres/pg_hba.conf`):

```
# Conexiones locales dentro del contenedor (psql)
local   all             all                                     trust

# Loopback IPv4 e IPv6
host    all             all             127.0.0.1/32            trust
host    all             all             ::1/128                 trust

# Acceso desde los servidores n8n (subred interna)
host    n8n_db          n8n_user        <SUBNET_N8N>/24         md5
```

Solo la subred `<SUBNET_N8N>/24` puede conectarse a `n8n_db` . Cualquier otra IP es rechazada.

Credenciales de la base de datos:

```
Base de datos:  n8n_db
Usuario:        n8n_user
Contraseña:     PostgresSQL
Puerto:         5432
```

Opción A — Usando el script automatizado (requiere sshpass en WSL/Linux):

```
cd Postgres/
bash deploy.sh
```

El script tiene 7 pasos (los demás tienen 5): incluye backup previo y restauración posterior automáticos.

Opción B — Instalación manual directa en el servidor (cuando no se puede usar deploy.sh):

```
ssh administrador@<IP_POSTGRES>

# ── PASO 0: BACKUP OBLIGATORIO si ya existen datos ──
# Ejecutar SIEMPRE antes de continuar si hay una instalación previa de PostgreSQL
if systemctl is-active --quiet postgresql; then
    sudo -u postgres pg_dump n8n_db > /tmp/n8n_db_backup.sql
    echo "Backup guardado en /tmp/n8n_db_backup.sql"
fi

# 1. Instalar Docker si no está presente
if ! command -v docker &> /dev/null; then
    curl -fsSL https://get.docker.com | sudo sh
    sudo systemctl enable docker
    sudo systemctl start docker
fi

# 2. Crear directorio de trabajo
sudo mkdir -p /opt/n8n-postgres
sudo chown administrador:administrador /opt/n8n-postgres

# 3. Crear el archivo docker-compose.yml
cat > /opt/n8n-postgres/docker-compose.yml << 'EOF'
```

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: n8n-postgres
    restart: always
    network_mode: host
    environment:
      POSTGRES_DB: n8n_db
      POSTGRES_USER: n8n_user
      POSTGRES_PASSWORD: PostgresSQL
      POSTGRES_HOST_AUTH_METHOD: md5
      POSTGRES_INITDB_ARGS: "--auth-host=md5 --auth-local=md5"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./pg_hba.conf:/etc/postgresql/pg_hba.conf:ro
    command: >
      postgres
      -c listen_addresses=*
      -c hba_file=/etc/postgresql/pg_hba.conf

volumes:
  postgres_data:
    name: n8n_postgres_data
EOF

# 4. Crear el archivo pg_hba.conf
cat > /opt/n8n-postgres/pg_hba.conf << 'EOF'
# TYPE  DATABASE        USER            ADDRESS                 METHOD

# Conexiones locales dentro del contenedor
local   all             all                                     trust

# Loopback IPv4 e IPv6
host    all             all             127.0.0.1/32            trust
host    all             all             ::1/128                 trust

# Acceso desde servidores n8n (subred interna)
host    n8n_db          n8n_user        <SUBNET_N8N>/24         md5
EOF

# 5. Detener PostgreSQL systemd previo si existe
sudo systemctl stop postgresql 2>/dev/null || true
sudo systemctl disable postgresql 2>/dev/null || true

# 6. Levantar el contenedor
cd /opt/n8n-postgres
sudo docker compose pull
sudo docker compose up -d

# 7. Esperar a que esté listo
echo "Esperando que PostgreSQL esté disponible..."
until sudo docker exec n8n-postgres pg_isready -U n8n_user 2>/dev/null; do
    sleep 2
done
echo "PostgreSQL listo."
```

```
# 8. Restaurar backup si existía (solo si se hizo en el paso 0)
if [ -f /tmp/n8n_db_backup.sql ]; then
    sudo docker exec -i n8n-postgres psql -U n8n_user -d n8n_db < /tmp/n8n_db_backup.sql
    echo "Datos restaurados correctamente."
fi
```

Verificación:

```
ssh administrador@<IP_POSTGRES>
sudo docker exec n8n-postgres pg_isready -U n8n_user
# Debe responder: /var/run/postgresql:5432 - accepting connections

sudo docker exec -it n8n-postgres psql -U n8n_user -d n8n_db -c "\dt"
# Lista las tablas de n8n
```

3.3 n8n Main 1 — `<IP_MAIN1>:5678`

Nodo activo. Sirve la interfaz web, procesa la API REST y gestiona los webhooks. El Load Balancer distribuye el tráfico entre Main 1 y Main 2 usando `ip_hash` (sesiones pegajosas por IP del cliente).

Contenedor desplegado (`Main 1/docker-compose.yml`):

```yaml
services:
  n8n:
    image: n8nio/n8n:2.8.4
    container_name: n8n-main1
    restart: always
    network_mode: host
    environment:
      - N8N_HOST=0.0.0.0
      - N8N_PORT=5678
      - EXECUTIONS_MODE=queue
      - QUEUE_BULL_REDIS_HOST=<IP_REDIS>
      - QUEUE_BULL_REDIS_PORT=6379
      - DB_TYPE=postgresdb
      - DB_POSTGRESDB_HOST=<IP_POSTGRES>
      - DB_POSTGRESDB_PORT=5432
      - DB_POSTGRESDB_DATABASE=n8n_db
      - DB_POSTGRESDB_USER=n8n_user
      - DB_POSTGRESDB_PASSWORD=PostgresSQL
      - N8N_ENCRYPTION_KEY=main1VMn8nArtics
      - N8N_SECURE_COOKIE=false
      - N8N_EDITOR_BASE_URL=https://<IP_LB>
      - WEBHOOK_URL=https://<IP_LB>
      - N8N_PROXY_HOPS=1
      - N8N_PROTOCOL=https
      - N8N_PUSH_BACKEND=websocket
    volumes:
      - n8n_data:/home/node/.n8n

volumes:
  n8n_data:
    name: n8n_main1_data
```

Explicación de las variables más importantes:

Variable	Valor	Por qué
`EXECUTIONS_MODE=queue`	`queue`	No ejecuta flujos aquí; los delega al Worker via Redis
`N8N_ENCRYPTION_KEY`	`main1VMn8nArtics`	Clave para cifrar credenciales en DB. **Debe ser igual en todos los nodos**
`N8N_SECURE_COOKIE=false`	`false`	La conexión interna nginx → n8n sigue siendo HTTP local; n8n no ve HTTPS directamente
`N8N_EDITOR_BASE_URL`	`https://<IP_LB>`	URL pública del editor (la del LB, con HTTPS)
`WEBHOOK_URL`	`https://<IP_LB>`	URL base pública para los webhooks generados por n8n
`N8N_PROXY_HOPS=1`	`1`	Indica que hay 1 proxy delante (Nginx), para leer correctamente `X-Forwarded-For`
`N8N_PROTOCOL=https`	`https`	Le indica a n8n que el protocolo público es HTTPS (necesario junto a `WEBHOOK_URL`)
`N8N_PUSH_BACKEND=websocket`	`websocket`	**Fix aplicado** — ver sección 6, Fix #3

> ⚠️ ***Importante:*** *Al levantar Main 1 por primera vez, n8n ejecuta las* ***migraciones de base de datos*** *automáticamente. Esperar mínimo 20 segundos antes de levantar el Worker.*

Opción A — Usando el script automatizado (requiere sshpass en WSL/Linux):

```
cd "Main 1/"
bash deploy.sh
```

Opción B — Instalación manual directa en el servidor (cuando no se puede usar deploy.sh):

```
ssh administrador@<IP_MAIN1>

# 1. Instalar Docker si no está presente
if ! command -v docker &> /dev/null; then
    curl -fsSL https://get.docker.com | sudo sh
    sudo systemctl enable docker
    sudo systemctl start docker
fi

# 2. Crear directorio de trabajo
sudo mkdir -p /opt/n8n-main1
sudo chown administrador:administrador /opt/n8n-main1

# 3. Crear el archivo docker-compose.yml
cat > /opt/n8n-main1/docker-compose.yml << 'EOF'
services:
  n8n:
    image: n8nio/n8n:2.8.4
    container_name: n8n-main1
    restart: always
    network_mode: host
    environment:
      - N8N_HOST=0.0.0.0
      - N8N_PORT=5678
      - EXECUTIONS_MODE=queue
      - QUEUE_BULL_REDIS_HOST=<IP_REDIS>
      - QUEUE_BULL_REDIS_PORT=6379
      - DB_TYPE=postgresdb
```

```
      - DB_POSTGRESDB_HOST=<IP_POSTGRES>
      - DB_POSTGRESDB_PORT=5432
      - DB_POSTGRESDB_DATABASE=n8n_db
      - DB_POSTGRESDB_USER=n8n_user
      - DB_POSTGRESDB_PASSWORD=PostgresSQL
      - N8N_ENCRYPTION_KEY=main1VMn8nArtics
      - N8N_SECURE_COOKIE=false
      - N8N_EDITOR_BASE_URL=https://<IP_LB>
      - WEBHOOK_URL=https://<IP_LB>
      - N8N_PROXY_HOPS=1
      - N8N_PROTOCOL=https
      - N8N_PUSH_BACKEND=websocket
    volumes:
      - n8n_data:/home/node/.n8n

volumes:
  n8n_data:
    name: n8n_main1_data
EOF

# 4. Detener servicio n8n systemd previo si existe
sudo systemctl stop n8n 2>/dev/null || true
sudo systemctl disable n8n 2>/dev/null || true

# 5. Levantar el contenedor
cd /opt/n8n-main1
sudo docker compose pull
sudo docker compose up -d

# 6. Esperar migraciones de DB (importante — no saltear)
echo "Esperando 20s para que n8n ejecute las migraciones de base de datos..."
sleep 20
sudo docker exec n8n-main1 wget -qO- http://localhost:5678/healthz
```

Verificación:

```
ssh administrador@<IP_MAIN1>
sudo docker exec n8n-main1 wget -qO- http://localhost:5678/healthz
# Debe responder: {"status":"ok"}

sudo docker logs n8n-main1 --tail 30
# No debe haber errores de conexión a Redis o PostgreSQL
```

3.4 n8n Worker — `<IP_WORKER>`

Ejecuta los flujos de trabajo. No tiene interfaz web. Escucha la cola Redis y ejecuta hasta 10 flujos simultáneamente.

Contenedor desplegado (`Worker/docker-compose.yml`):

```yaml
services:
  n8n-worker:
    image: n8nio/n8n:2.8.4
    container_name: n8n-worker
    restart: always
    network_mode: host
```

```yaml
    command: worker --concurrency=10
    environment:
      - EXECUTIONS_MODE=queue
      - QUEUE_BULL_REDIS_HOST=<IP_REDIS>
      - QUEUE_BULL_REDIS_PORT=6379
      - DB_TYPE=postgresdb
      - DB_POSTGRESDB_HOST=<IP_POSTGRES>
      - DB_POSTGRESDB_PORT=5432
      - DB_POSTGRESDB_DATABASE=n8n_db
      - DB_POSTGRESDB_USER=n8n_user
      - DB_POSTGRESDB_PASSWORD=PostgresSQL
      - N8N_ENCRYPTION_KEY=main1VMn8nArtics
      - N8N_DISABLE_UI=true
    volumes:
      - n8n_worker_data:/home/node/.n8n

volumes:
  n8n_worker_data:
    name: n8n_worker_data
```

Diferencias clave respecto a Main:

- `command: worker --concurrency=10` — arranca en modo worker, no como servidor web
- `N8N_DISABLE_UI=true` — no expone interfaz web (ahorra recursos)
- No tiene `N8N_HOST`, `N8N_PORT`, `N8N_EDITOR_BASE_URL` ni `WEBHOOK_URL`
- No tiene `N8N_PROXY_HOPS` (no hay proxy delante)

Opción A — Usando el script automatizado (requiere sshpass en WSL/Linux):

```bash
cd Worker/
bash deploy.sh
```

Opción B — Instalación manual directa en el servidor (cuando no se puede usar deploy.sh):

```bash
ssh administrador@<IP_WORKER>

# 1. Instalar Docker si no está presente
if ! command -v docker &> /dev/null; then
    curl -fsSL https://get.docker.com | sudo sh
    sudo systemctl enable docker
    sudo systemctl start docker
fi

# 2. Crear directorio de trabajo
sudo mkdir -p /opt/n8n-worker
sudo chown administrador:administrador /opt/n8n-worker

# 3. Crear el archivo docker-compose.yml
cat > /opt/n8n-worker/docker-compose.yml << 'EOF'
services:
  n8n-worker:
    image: n8nio/n8n:2.8.4
    container_name: n8n-worker
    restart: always
    network_mode: host
    command: worker --concurrency=10
```

```
    environment:
      - EXECUTIONS_MODE=queue
      - QUEUE_BULL_REDIS_HOST=<IP_REDIS>
      - QUEUE_BULL_REDIS_PORT=6379
      - DB_TYPE=postgresdb
      - DB_POSTGRESDB_HOST=<IP_POSTGRES>
      - DB_POSTGRESDB_PORT=5432
      - DB_POSTGRESDB_DATABASE=n8n_db
      - DB_POSTGRESDB_USER=n8n_user
      - DB_POSTGRESDB_PASSWORD=PostgresSQL
      - N8N_ENCRYPTION_KEY=main1VMn8nArtics
      - N8N_DISABLE_UI=true
    volumes:
      - n8n_worker_data:/home/node/.n8n

volumes:
  n8n_worker_data:
    name: n8n_worker_data
EOF

# 4. Detener servicio n8n systemd previo si existe
sudo systemctl stop n8n 2>/dev/null || true
sudo systemctl disable n8n 2>/dev/null || true

# 5. Levantar el contenedor
cd /opt/n8n-worker
sudo docker compose pull
sudo docker compose up -d

# 6. Verificar logs (esperar ~10 segundos)
sleep 10
sudo docker logs n8n-worker --tail 20
```

Verificación:

```
ssh administrador@<IP_WORKER>
sudo docker logs n8n-worker --tail 30
# Debe mostrar: "Worker ID: ... registered" y "Listening for jobs"
# No debe haber errores de conexión
```

3.5 n8n Main 2 — `<IP_MAIN2>:5678`

Nodo activo secundario. Recibe aproximadamente el 50% del tráfico del Load Balancer junto con Main 1. Ambos nodos comparten la misma base de datos PostgreSQL y la misma cola Redis, por lo que operan de forma coordinada.

> *El `ip_hash` de Nginx garantiza que un mismo cliente siempre sea dirigido al mismo nodo Main durante su sesión, evitando problemas con el canal WebSocket `/rest/push`.*

Diferencias respecto a Main 1:

- `container_name: n8n-main2`
- Volumen: `n8n_main2_data` (separado del de Main 1)

Contenedor desplegado (`Main 2/docker-compose.yml`):

```yaml
services:
  n8n:
    image: n8nio/n8n:2.8.4
    container_name: n8n-main2
    restart: always
    network_mode: host
    environment:
      - N8N_HOST=0.0.0.0
      - N8N_PORT=5678
      - EXECUTIONS_MODE=queue
      - QUEUE_BULL_REDIS_HOST=<IP_REDIS>
      - QUEUE_BULL_REDIS_PORT=6379
      - DB_TYPE=postgresdb
      - DB_POSTGRESDB_HOST=<IP_POSTGRES>
      - DB_POSTGRESDB_PORT=5432
      - DB_POSTGRESDB_DATABASE=n8n_db
      - DB_POSTGRESDB_USER=n8n_user
      - DB_POSTGRESDB_PASSWORD=PostgresSQL
      - N8N_ENCRYPTION_KEY=main1VMn8nArtics
      - N8N_SECURE_COOKIE=false
      - N8N_EDITOR_BASE_URL=https://<IP_LB>
      - WEBHOOK_URL=https://<IP_LB>
      - N8N_PROXY_HOPS=1
      - N8N_PROTOCOL=https
      - N8N_PUSH_BACKEND=websocket
    volumes:
      - n8n_data:/home/node/.n8n

volumes:
  n8n_data:
    name: n8n_main2_data
```

Opción A — Usando el script automatizado (requiere sshpass en WSL/Linux):

```bash
cd "Main 2/"
bash deploy.sh
```

Opción B — Instalación manual directa en el servidor (cuando no se puede usar deploy.sh):

```bash
ssh administrador@<IP_MAIN2>

# 1. Instalar Docker si no está presente
if ! command -v docker &> /dev/null; then
    curl -fsSL https://get.docker.com | sudo sh
    sudo systemctl enable docker
    sudo systemctl start docker
fi

# 2. Crear directorio de trabajo
sudo mkdir -p /opt/n8n-main2
sudo chown administrador:administrador /opt/n8n-main2

# 3. Crear el archivo docker-compose.yml
cat > /opt/n8n-main2/docker-compose.yml << 'EOF'
services:
```

```
  n8n:
    image: n8nio/n8n:2.8.4
    container_name: n8n-main2
    restart: always
    network_mode: host
    environment:
      - N8N_HOST=0.0.0.0
      - N8N_PORT=5678
      - EXECUTIONS_MODE=queue
      - QUEUE_BULL_REDIS_HOST=<IP_REDIS>
      - QUEUE_BULL_REDIS_PORT=6379
      - DB_TYPE=postgresdb
      - DB_POSTGRESDB_HOST=<IP_POSTGRES>
      - DB_POSTGRESDB_PORT=5432
      - DB_POSTGRESDB_DATABASE=n8n_db
      - DB_POSTGRESDB_USER=n8n_user
      - DB_POSTGRESDB_PASSWORD=PostgresSQL
      - N8N_ENCRYPTION_KEY=main1VMn8nArtics
      - N8N_SECURE_COOKIE=false
      - N8N_EDITOR_BASE_URL=https://<IP_LB>
      - WEBHOOK_URL=https://<IP_LB>
      - N8N_PROXY_HOPS=1
      - N8N_PROTOCOL=https
      - N8N_PUSH_BACKEND=websocket
    volumes:
      - n8n_data:/home/node/.n8n

volumes:
  n8n_data:
    name: n8n_main2_data
EOF

# 4. Detener servicio n8n systemd previo si existe
sudo systemctl stop n8n 2>/dev/null || true
sudo systemctl disable n8n 2>/dev/null || true

# 5. Levantar el contenedor
cd /opt/n8n-main2
sudo docker compose pull
sudo docker compose up -d

# 6. Esperar y verificar
sleep 15
sudo docker exec n8n-main2 wget -qO- http://localhost:5678/healthz
```

Verificación:

```
ssh administrador@<IP_MAIN2>
sudo docker exec n8n-main2 wget -qO- http://localhost:5678/healthz
# Debe responder: {"status":"ok"}

sudo docker logs n8n-main2 --tail 20
```

3.6 Load Balancer — Nginx HTTPS — `<IP_LB>:443`

El Load Balancer es el único punto de entrada público. **No se despliega con Docker** — Nginx se instala directamente en el servidor. Termina SSL y balancea el tráfico entre Main 1 y Main 2.

Instalación de Nginx

```
ssh administrador@<IP_LB>
sudo apt-get update
sudo apt-get install -y nginx openssl
```

Generación del certificado SSL autofirmado

```
sudo mkdir -p /etc/nginx/ssl
sudo openssl req -x509 -nodes -days 3650 -newkey rsa:2048 \
  -keyout /etc/nginx/ssl/n8n.key \
  -out /etc/nginx/ssl/n8n.crt \
  -subj "/C=CO/ST=Bogota/L=Bogota/O=Artics/CN=<IP_LB>"

# Verificar que se crearon
ls -la /etc/nginx/ssl/
```

> *El certificado dura 10 años (`-days 3650`). Al acceder por primera vez desde el navegador, éste mostrará un aviso de certificado no confiable — es esperado para un cert autofirmado. Aceptar la excepción una sola vez.*

Configuración final de Nginx (`Load Balancer/nginx.conf`)

Esta es la configuración **final y funcional** del archivo. Incorpora todos los fixes aplicados: HTTPS, upstream con ambos nodos, `ip_hash` para sesiones pegajosas y bloque separado para WebSocket.

Crear el archivo `/etc/nginx/sites-enabled/default`:

```
upstream n8n_backend {
    ip_hash;  # sesiones pegajosas: mismo IP siempre va al mismo main
    server <IP_MAIN1>:5678;  # Main 1
    server <IP_MAIN2>:5678;  # Main 2
}

server {
    listen 80;
    # Redirigir todo el tráfico HTTP a HTTPS
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    ssl_certificate /etc/nginx/ssl/n8n.crt;
    ssl_certificate_key /etc/nginx/ssl/n8n.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    access_log /var/log/nginx/n8n_access.log;

    # ── Canal de push WebSocket ──
    # ip_hash garantiza que el WS siempre llega al mismo main que inició la sesión
    location /rest/push {
        proxy_pass http://n8n_backend;
```

```
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-Host $host;
        proxy_set_header X-Forwarded-Port 443;
        proxy_buffering off;
        proxy_cache off;
        proxy_read_timeout 86400s;
        proxy_send_timeout 86400s;
        proxy_connect_timeout 60s;
    }

    # ── Resto de peticiones HTTP ──
    location / {
        proxy_pass http://n8n_backend;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "keep-alive";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-Host $host;
        proxy_set_header X-Forwarded-Port 443;
        proxy_read_timeout 300;
        proxy_connect_timeout 300;
        proxy_send_timeout 300;
        proxy_hide_header X-Frame-Options;
    }
}
```

Activar y recargar Nginx

```
# Verificar sintaxis
sudo nginx -t

# Recargar
sudo systemctl reload nginx
sudo systemctl enable nginx
```

> ***Nota:*** *La configuración se escribe directamente en* `sites-enabled/default` *(como hace el script* `fix-main2-lb.js`*). Si se prefiere usar* `sites-available/n8n` *con symlink, ajustar los paths en consecuencia.*

---

4. Despliegue automatizado

Hay dos formas de desplegar toda la infraestructura de una sola vez:

Opción A — Script Bash (`deploy-all.sh`)

Requiere `sshpass`. Ejecuta los `deploy.sh` individuales en el orden correcto con waits entre pasos.

```
# Desde WSL o Linux, en el directorio raíz del repositorio:
chmod +x deploy-all.sh
./deploy-all.sh
```

Tiempos de espera entre servicios:

- Tras Redis: 5 segundos
- Tras PostgreSQL: 10 segundos (para que esté listo para aceptar conexiones)
- Tras Main 1: 15 segundos (para que ejecute las migraciones de DB)
- Tras Worker: 5 segundos

Opción B — Script Node.js (`deploy-nodes.js`) — Recomendado

Versión más robusta. Además del despliegue, gestiona backup/restauración de PostgreSQL y hace health checks reales.

```
# Desde cualquier entorno con Node.js:
npm install
node deploy-nodes.js
```

Ventajas sobre el bash:

- Hace `pg_dump` automático antes de parar PostgreSQL y restaura los datos en el nuevo contenedor
- Si el backup ya existe en `/tmp/n8n_db_backup.sql` (run anterior), lo reutiliza
- Aborta el despliegue si el backup falla (para no perder datos)
- Muestra un resumen final con el estado de cada servidor

---

5. Variables de entorno críticas

Regla de oro: `N8N_ENCRYPTION_KEY`

```
N8N_ENCRYPTION_KEY=main1VMn8nArtics
```

Esta clave **debe ser idéntica** en Main 1, Main 2 y Worker. Si se cambia después de que ya existen credenciales guardadas, todas las credenciales en la base de datos quedarán ilegibles y n8n fallará al intentar usarlas.

Tabla de variables por servicio

Variable	Main 1	Main 2	Worker
N8N_HOST	0.0.0.0	0.0.0.0	—
N8N_PORT	5678	5678	—
EXECUTIONS_MODE	queue	queue	queue
QUEUE_BULL_REDIS_HOST	<IP_REDIS>	<IP_REDIS>	<IP_REDIS>
N8N_ENCRYPTION_KEY	main1VMn8nArtics	main1VMn8nArtics	main1VMn8nArtics
N8N_SECURE_COOKIE	false	false	—
N8N_EDITOR_BASE_URL	https://<IP_LB>	https://<IP_LB>	—
WEBHOOK_URL	https://<IP_LB>	https://<IP_LB>	—
N8N_PROXY_HOPS	1	1	—
N8N_PROTOCOL	https	https	—

N8N_PUSH_BACKEND	websocket	websocket	—
N8N_DISABLE_UI	—	—	true

---

6. Bugs encontrados y fixes aplicados

Esta sección documenta los problemas reales que se encontraron durante la instalación inicial y los fixes que se aplicaron para resolverlos. Los fixes están incorporados en las configuraciones actuales del repositorio.

Fix #1 — Activación de logs para diagnóstico (`enable-debug.js`)

Problema: Aparecían errores 403 intermitentes en la UI de n8n. No había información suficiente en los logs para determinar el origen (¿Nginx? ¿n8n? ¿un WAF externo?).

Solución aplicada:

1. En Nginx: se añadió `access_log /var/log/nginx/n8n_debug.log` y `error_log ... warn` al site config
2. En n8n Main 1: se añadió `N8N_LOG_LEVEL=verbose` al docker-compose

Cómo usar este script si se necesita diagnóstico:

```
node enable-debug.js
# Luego, para ver los logs:
ssh administrador@<IP_LB>
sudo tail -50 /var/log/nginx/n8n_debug.log

ssh administrador@<IP_MAIN1>
sudo docker logs n8n-main1 -f
```

> *Este fix es **temporal para diagnóstico**. En producción normal no se deja `verbose` activo porque genera demasiado volumen de logs.*

Fix #2 — Log format mejorado en Nginx (`fix-logging.js`)

Problema: El log estándar de Nginx solo mostraba el código HTTP final. No permitía ver si el 403 lo generaba Nginx o si venía del upstream (n8n). Esto dificultaba saber si era un problema del proxy o de la aplicación.

Solución aplicada: Se definió un `log_format` personalizado que incluye `$upstream_status` separado del `$status` de Nginx:

```
log_format n8nlog '$remote_addr [$time_local] "$request" $status upstream:$upstream_status bytes:$bytes_sent';
```

Con esto, un log como `"PATCH /rest/workflows/123" 403 upstream:403` confirma que el 403 viene de n8n, mientras que `"PATCH ..." 403 upstream:-` indicaría que Nginx lo rechaza antes de llegar al backend.

Adicionalmente se cambió el log level de `verbose` a `debug` en n8n (valor válido en n8n 2.x, donde `verbose` fue deprecado).

Fix #3 — Canal WebSocket `/rest/push` (FIX CRÍTICO) (`fix-push-websocket.js`)

Problema: La interfaz de n8n mostraba errores de conexión al intentar ver el estado de ejecuciones en tiempo real. El editor se desconectaba repetidamente. Los síntomas eran:

- La UI mostraba "Connection lost" o el estado de los nodos no se actualizaba
- Errores en consola del navegador en el endpoint `/rest/push`

Causa raíz (dos problemas combined):

1. **En Nginx:** El `location /` general enviaba `Connection: upgrade` para todas las peticiones, pero las peticiones HTTP normales (no WebSocket) no deben tener ese header. Además, Nginx tenía un `proxy_read_timeout` de 300 segundos, lo cual cerraba la conexión WebSocket persistente de `/rest/push` cada 5 minutos.
2. **En n8n:** Por defecto, n8n 2.x usa Server-Sent Events (SSE) como backend de push. SSE no es compatible con todas las configuraciones de proxy. Hay que forzarlo explícitamente a WebSocket.

Solución aplicada:

En Nginx — separar `/rest/push` en su propio bloque `location`:

```
# /rest/push: WebSocket puro, sin buffering, timeout de 24 horas
location /rest/push {
    proxy_buffering off;
    proxy_cache off;
    proxy_read_timeout 86400s;   # 24 horas
    proxy_send_timeout 86400s;
    proxy_set_header Connection "upgrade";
    ...
}

# Resto: HTTP normal con Connection: keep-alive
location / {
    proxy_set_header Connection "keep-alive";
    proxy_read_timeout 300;      # 5 minutos es suficiente para HTTP
    ...
}
```

En n8n Main 1 y Main 2 — forzar WebSocket:

```
- N8N_PUSH_BACKEND=websocket
```

Estado: ✅ Este fix está incorporado en los `docker-compose.yml` actuales de Main 1 y Main 2, y en la configuración de Nginx de este manual.

Fix #4 — Investigación de bloqueo por WAF (`fix-https.js`)

Problema: Se sospechaba que el método HTTP `PATCH` (usado por n8n para actualizar workflows) estaba siendo bloqueado con 403 por un WAF (Web Application Firewall) en la red de Positiva, ya que dicho método no es estándar en muchas políticas de WAF.

Hipótesis: Si el WAF opera en capa HTTP pero no inspecciona tráfico HTTPS/SSL, cifrar el tráfico podría solucionar el bloqueo.

Qué hace el script:

1. Genera un certificado SSL autofirmado en el Load Balancer (`openssl req -x509 ...`)
2. Reconfigura Nginx para escuchar en HTTPS (puerto 443) con redirect 80→443
3. Prueba si `PATCH` pasa por HTTPS desde el script (sin pasar por navegador)
4. Si el PATCH pasa: actualiza `N8N_EDITOR_BASE_URL` y `WEBHOOK_URL` a `https://` en Main 1 y Main 2

Resultado / Estado actual: ✅ **HTTPS está activo.** El entorno funciona en HTTPS (puerto 443) con certificado autofirmado. El WAF no intercepta el tráfico SSL, por lo que `PATCH` pasa sin bloqueo. Los nodos Main 1 y Main 2 tienen `N8N_EDITOR_BASE_URL=https://...` y `N8N_PROTOCOL=https`. La activación final se realizó con el script `fix-main2-lb.js` (Fix #5).

Para volver a ejecutar si se necesita recrear la config HTTPS en el LB:

```
node fix-https.js
```

Fix #5 — Activación de Main 2 como nodo activo + upstream HTTPS (`fix-main2-lb.js`)

Problema: Main 2 estaba levantado pero no recibía tráfico. El Load Balancer apuntaba únicamente a Main 1. Además, Main 2 no tenía `N8N_PROXY_HOPS=1` ni `N8N_PROTOCOL=https` , por lo que si se lo activaba manualmente, funcionaba de forma incorrecta.

Solución aplicada (3 cambios en paralelo):

1. **Nginx:** Se agregó un bloque `upstream n8n_backend` con `ip_hash` que incluye a Main 1 y Main 2. Se migró el servidor a HTTPS (port 443) con certificado SSL autofirmado y redirect HTTP→HTTPS. El `ip_hash` garantiza que cada cliente siempre llega al mismo nodo Main, lo cual es crítico para que el WebSocket `/rest/push` no cambie de nodo entre peticiones.
2. **Main 1:** Se actualizaron `N8N_EDITOR_BASE_URL` y `WEBHOOK_URL` de `http://` a `https://` , y se agregó `N8N_PROTOCOL=https` .
3. **Main 2:** Mismos cambios que Main 1, más la adición de `N8N_PROXY_HOPS=1` que le faltaba.

Estado: ✅ Incorporado en los `docker-compose.yml` actuales de Main 1, Main 2 y en `Load Balancer/nginx.conf` .

Para re-aplicar todos estos cambios de una vez contra los servidores:

```
node fix-main2-lb.js
```

---

7. Verificación post-instalación

Ejecutar estas verificaciones en orden tras completar el despliegue completo.

7.1 Verificar que todos los contenedores están corriendo

```
# Redis
ssh administrador@<IP_REDIS> "sudo docker compose -f /opt/n8n-redis/docker-compose.yml ps"

# PostgreSQL
ssh administrador@<IP_POSTGRES> "sudo docker compose -f /opt/n8n-postgres/docker-compose.yml ps"

# Main 1
ssh administrador@<IP_MAIN1> "sudo docker compose -f /opt/n8n-main1/docker-compose.yml ps"

# Worker
ssh administrador@<IP_WORKER> "sudo docker compose -f /opt/n8n-worker/docker-compose.yml ps"

# Main 2
ssh administrador@<IP_MAIN2> "sudo docker compose -f /opt/n8n-main2/docker-compose.yml ps"
```

Todos deben mostrar estado `Up` .

7.2 Verificar conectividad de servicios

```
# Redis responde
ssh administrador@<IP_REDIS> "sudo docker exec n8n-redis redis-cli ping"
# Esperado: PONG

# PostgreSQL acepta conexiones
ssh administrador@<IP_POSTGRES> "sudo docker exec n8n-postgres pg_isready -U n8n_user"
# Esperado: /var/run/postgresql:5432 - accepting connections
```

```
# n8n Main 1 healthcheck
ssh administrador@<IP_MAIN1> "sudo docker exec n8n-main1 wget -qO- http://localhost:5678/healthz"
# Esperado: {"status":"ok"}
```

7.3 Verificar acceso desde el Load Balancer

```
# Desde la máquina local, o haciendo curl pasando por el LB (certificado autofirmado, usar -k):
curl -k https://<IP_LB>/healthz
# Esperado: {"status":"ok"}

# Verificar ambos nodos responden
curl -k https://<IP_LB>/healthz  # puede llegar a Main 1 o Main 2
ssh administrador@<IP_MAIN2> "sudo docker exec n8n-main2 wget -qO- http://localhost:5678/healthz"
# Ambos deben responder: {"status":"ok"}

# Verificar Nginx
ssh administrador@<IP_LB> "sudo systemctl status nginx"
```

7.4 Verificar el Worker consume jobs de Redis

```
ssh administrador@<IP_WORKER> "sudo docker logs n8n-worker --tail 20"
# Debe mostrar líneas como:
# "Worker ID: X-Y-Z registered"
# "Listening to queue"
# No debe haber errores de conexión a Redis o PostgreSQL
```

7.5 Verificar desde el navegador

1. Abrir `https://<IP_LB>` (el navegador mostrará aviso de certificado autofirmado — aceptar la excepción una vez)
2. Hacer login con las credenciales de n8n
3. Abrir un workflow y ejecutarlo manualmente
4. Verificar que el estado de los nodos se actualiza en tiempo real (valida el fix WebSocket)
5. Guardar el workflow con `PATCH` (valida que el WAF no bloquea el método)

---

8. Comandos de operación diaria

Ver logs en tiempo real

```
# n8n Main 1
ssh administrador@<IP_MAIN1> "sudo docker logs n8n-main1 -f"

# n8n Worker
ssh administrador@<IP_WORKER> "sudo docker logs n8n-worker -f"

# Nginx (access log)
ssh administrador@<IP_LB> "sudo tail -f /var/log/nginx/n8n_debug.log"

# Nginx (error log)
ssh administrador@<IP_LB> "sudo tail -f /var/log/nginx/n8n_error.log"
```

Reiniciar servicios

```
# Reiniciar un contenedor individual
ssh administrador@<IP_MAIN1> "cd /opt/n8n-main1 && sudo docker compose restart"

# Reiniciar Nginx
ssh administrador@<IP_LB> "sudo systemctl reload nginx"
```

Ver jobs activos en Redis

```
ssh administrador@<IP_REDIS> "sudo docker exec n8n-redis redis-cli monitor"
# Ctrl+C para salir
```

Backup manual de PostgreSQL

```
ssh administrador@<IP_POSTGRES>
sudo docker exec n8n-postgres pg_dump -U n8n_user n8n_db > /tmp/backup_$(date +%Y%m%d_%H%M%S).sql
```

Desactivar un nodo Main temporalmente

Si Main 1 (o Main 2) falla y hay que sacarlo del upstream mientras se repara:

```
ssh administrador@<IP_LB>
sudo nano /etc/nginx/sites-enabled/default
# En el bloque upstream, comentar el nodo que falla:
#   upstream n8n_backend {
#       ip_hash;
#       server <IP_MAIN1>:5678;  # <-- comentar esta línea si Main 1 está caído
#       server <IP_MAIN2>:5678;
#   }

sudo nginx -t
sudo systemctl reload nginx
```

Cuando el nodo sea reparado, descomentar la línea y recargar nginx.

Apéndice — IPs de referencia rápida

Servicio	IP	Puerto	Acceso
UI n8n (acceso público)		443	`https://<IP_LB>`
n8n Main 1 (directo)		5678	`http://<IP_MAIN1>:5678`
n8n Main 2 (directo)		5678	`http://<IP_MAIN2>:5678`
Redis		6379	Solo interno
n8n Worker		—	Solo SSH
PostgreSQL		5432	Solo subred /24
Microservicios		varios	Equipo Camila

Clave de cifrado n8n (no cambiar nunca sin migrar credenciales):

```
N8N_ENCRYPTION_KEY=main1VMn8nArtics
```